

Assignment 3

Smart Contract Implementation, Unit Testing, and dApp Wireframe

Assignment Type	Group project
Submission Mode	One group submission based on the Assignment 2 white paper
Suggested Timeline	2 weeks
Primary Outputs	Smart contract code, unit tests, dApp wireframe, and brief technical documentation

Overview

This assignment builds on the white paper developed in Assignment 2. Each group will translate its proposed Web3 project into an implementable prototype by submitting core smart contract code, a unit testing suite, and a wireframe for the corresponding decentralized application (dApp). The goal is to demonstrate that the project vision described in the white paper can be expressed as a clear on-chain design with testable logic and a usable front-end workflow.

By the end of this assignment, students should be able to: map a white paper into one or more core smart contracts; justify key on-chain functions, permissions, and state transitions; validate implementation correctness with meaningful tests; and communicate the intended user experience through a structured dApp wireframe.

Requirements

1. Smart Contract Implementation

- Implement the core smart contract logic for the project proposed in your Assignment 2 white paper. The design should be clearly aligned with the project scope, token model, governance logic, NFT/RWA mechanism, or transaction workflow described in the white paper.
- You may submit one contract or multiple contracts, depending on the architecture of your project. All contracts must compile successfully.
- The code should include meaningful comments, clear naming, events where appropriate, access control logic, and basic input validation or error handling.
- Your submission should identify the purpose of each contract and explain how the contracts interact at a system level.

2. Unit Testing

- Provide a unit testing suite for the submitted smart contract code. Tests should cover core functions, expected state changes, and important failure cases.
- At minimum, tests should demonstrate successful execution of the main user flows and include negative tests for invalid or unauthorized behavior where relevant.
- You may use Hardhat, Foundry, Truffle, Remix testing, or another appropriate testing framework, provided the test procedure is clearly documented.

3. dApp Wireframe

- Submit a wireframe for the front end of your project. The wireframe should show the major pages or screens of the dApp and the expected user journey.
- At minimum, include the core user interactions such as wallet connection, key transaction or participation flow, user dashboard or asset view, and administrative/governance interaction if applicable.
- The wireframe may be created using Figma, PowerPoint, PDF mockups, hand-drawn sketches converted to PDF, or another clear visual format.

4. Brief Technical Documentation

- Include a short README or report that explains how to compile the contracts, run the tests, and interpret the wireframe.
- Document any assumptions, libraries, or external dependencies used in the project.

Suggested Submission Structure

- Source code folder containing all Solidity contracts or equivalent smart contract files.
- Test folder containing executable unit tests and any required scripts/configuration files.
- One README or short report in PDF format describing project logic, setup, and testing steps.
- Wireframe file(s) in PDF, PNG, PPT, or Figma export format.
- A short note linking the implementation back to the major features promised in the white paper.

Important: Your implementation does not need to be production-ready, but it must be logically consistent, testable, and clearly connected to the white paper submitted in Assignment 2.

Deliverables

- Smart contract source code.
- Unit testing code and evidence of test execution.
- dApp wireframe showing major screens and user flow.
- A brief technical note or README explaining the implementation and testing process.

Recommended evidence to include: compiler version, framework used, test command, screenshots or logs of successful test runs, and labels on the wireframe showing the purpose of each screen.

Marking Rubric

Category	Excellent (10)	Good (7-9)	Satisfactory (4-6)	Needs Improvement (1-3)
White Paper Alignment & Design Logic	Implementation strongly reflects the white paper; on-chain logic is coherent and well justified.	Good alignment with the white paper; only minor gaps in design explanation.	Basic alignment is visible, but important connections or rationale are underdeveloped.	Weak connection to the white paper or unclear project logic.
Smart Contract Functionality	Core features are implemented correctly and compile successfully with clear structure.	Most required features work correctly; minor issues do not affect core functionality.	Some core functionality is present, but the implementation is incomplete or inconsistent.	Contracts are poorly implemented, incomplete, or fail to compile.
Code Quality & Security Considerations	Code is clean, readable, commented, and demonstrates sound access control and validation.	Code is generally readable with reasonable structure; some security or clarity details are missing.	Code quality is uneven; comments, structure, or validation are limited.	Code is difficult to follow and lacks basic safeguards or organization.
Unit Testing	Tests are comprehensive, cover core flows and failure cases, and clearly validate correctness.	Tests cover main functionality with only minor gaps in depth or coverage.	Some tests are provided, but important cases are missing or weakly justified.	Few or no meaningful tests are provided.
dApp Wireframe & User Flow	Wireframe is clear, complete, and presents a strong end-to-end user experience.	Wireframe communicates major screens and flows with minor missing details.	Wireframe is understandable but limited in depth, labeling, or flow logic.	Wireframe is incomplete, unclear, or disconnected from the project.
Overall Presentation & Documentation	Submission is professional, well organized, and easy to evaluate.	Submission is organized and readable with minor presentation issues.	Submission is understandable but lacks polish or clear organization.	Submission is poorly organized, difficult to follow, or missing documentation.

Grading Breakdown

Smart Contract Implementation	35%
Unit Testing & Validation	25%
dApp Wireframe & User Flow	15%
White Paper Alignment & Design Rationale	15%
Documentation & Presentation	10%

Deadline

The submission deadline is 2 weeks. Late submissions may be penalized according to course policy.